

LabReSiD25 - Hands-On 1

Davide Ferrara - 518629

1 Esercizio 1: Comunicazione UDP

1.1 Obiettivo

Usando i Linux Socket scrivere un server (`server_udp.c`) ed un client (`client_udp.c`) che realizzino una comunicazione bidirezionale basata su UDP.

UDP (User Datagram Protocol) è un protocollo non orientato alla connessione del modello TCP/IP. A differenza di TCP non richiede `connect()/listen()/accept()` ma utilizza direttamente `recvfrom()` e `sendto()`.

1.2 Server UDP

Includo le librerie necessarie per la comunicazione di rete.

```
1 #include <arpa/inet.h>    // inet_ntoa, htons, ntohs
2 #include <stddef.h>       // size_t
3 #include <stdio.h>        // printf, perror
4 #include <stdlib.h>       // exit
5 #include <string.h>       // memset, strlen
6 #include <sys/socket.h>   // socket, bind, recvfrom,
    sendto
7 #include <unistd.h>       // close
```

Listing 1: server_udp.c - Include

Definisco le costanti per la porta e la dimensione del buffer.

```
1 #define PORT 8080
2 #define BUF_SIZE 1024
```

Listing 2: server_udp.c - Costanti

Creo la struttura `sockaddr_in` per IPv4 contenente indirizzo e porta.

- `AF_INET`: famiglia di indirizzi IPv4

- `htons()`: converte la porta in formato di rete (big-endian)
- `INADDR_ANY = 0.0.0.0`: il server ascolta su TUTTE le interfacce di rete

```

1 struct sockaddr_in addr_in;
2 memset(&addr_in, 0, sizeof(addr_in));
3 addr_in.sin_family = AF_INET;
4 addr_in.sin_port = htons(PORT);
5 addr_in.sin_addr.s_addr = INADDR_ANY;

```

Listing 3: server_udp.c - `sockaddr_in`

Creo il socket di tipo `SOCK_DGRAM` (datagram) per UDP. La `socket()` restituisce un file descriptor. Controllo con `-1` per gestire gli errori.

```

1 int server_fd = socket(AF_INET, SOCK_DGRAM, 0);
2 if (server_fd == -1) {
3     perror("Creazione socket fallita!");
4     exit(1);
5 }

```

Listing 4: server_udp.c - `socket()`

Il `bind()` associa il socket all'indirizzo e porta specificati nella struttura. Il server rimane in ascolto su quella porta.

```

1 if (bind(server_fd, (struct sockaddr *)&addr_in, sizeof(
    addr_in)) == -1) {
2     perror("Errore nel bind");
3     close(server_fd);
4     exit(1);
5 }
6 printf("Server UDP in ascolto sulla porta %d...\n", PORT
    );

```

Listing 5: server_udp.c - `bind()`

Alloco il buffer per ricevere i dati e la struttura `sockaddr_in` per memorizzare le informazioni sul client.

```

1 char buf[BUF_SIZE];
2 struct sockaddr_in client_addr;
3 socklen_t client_len = sizeof(client_addr);

```

Listing 6: server_udp.c - Buffer

Ciclo infinito: `recvfrom()` è bloccante e riceve datagrammi UDP, restituisce anche l'indirizzo del client e dopo aver stampato il messaggio, rispondo con `sendto()`.

```
1 while (1) {
2     ssize_t n = recvfrom(server_fd, buf, sizeof(buf) -
3         1, 0,
4         (struct sockaddr *)&client_addr,
5         &client_len);
6     if (n > 0) {
7         buf[n] = '\0';
8         printf("Ricevuto da %s:%d: %s\n",
9             inet_ntoa(client_addr.sin_addr),
10            ntohs(client_addr.sin_port), buf);
11         char *msg = "pong\n";
12         sendto(server_fd, msg, strlen(msg), 0,
13             (struct sockaddr *)&client_addr,
14             client_len);
15     }
16 }
```

Listing 7: server_udp.c - recvfrom/sendto

1.3 Client UDP

Struttura dati analoga al server, ma per il server remoto, uso 127.0.0.1 (localhost) come indirizzo del server.

```
1 struct sockaddr_in server_addr;
2 memset(&server_addr, 0, sizeof(server_addr));
3 server_addr.sin_family = AF_INET;
4 server_addr.sin_port = htons(PORT);
5 server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

Listing 8: client_udp.c - sockaddr_in

Invio il messaggio "ping" al server con `sendto()`, ricevo la risposta con `recvfrom()`. Il NULL nei parametri finali indica che non mi serve l'indirizzo del server (lo conosco già).

`sleep(1)` invece pausa il ciclo di 1 secondo.

```
1 char buf[BUF_SIZE];
2 while (1) {
3     sendto(client_fd, "ping\n", 5, 0,
4             (struct sockaddr *)&server_addr, sizeof(
5                 server_addr));
6     printf("Inviato a %s:%d: ping\n", ADDR, PORT);
7
8     ssize_t n = recvfrom(client_fd, buf, sizeof(buf) -
9                           1, 0, NULL, NULL);
10    if (n > 0) {
11        buf[n] = '\0';
12        printf("Ricevuto da %s:%d: %s\n", ADDR, PORT,
13              buf);
14    }
15    sleep(1);
16 }
```

Listing 9: client_udp.c - Invio/Ricezione

Chiudo il file descriptor.

```
1 close(client_fd);
2 return 0;
```

Listing 10: client_udp.c - Chiusura

1.4 Differenze Client/Server

Il server effettua il `bind()` sulla porta per rimanere in ascolto. Il client invia direttamente al server senza `bind`. La restante logica è praticamente uguale.

1.5 Output di esempio

```
1 Server UDP in ascolto sulla porta 8080...
2 Ricevuto da 127.0.0.1:37698: ping
3 Ricevuto da 127.0.0.1:37698: ping
4 Ricevuto da 127.0.0.1:37698: ping
```

Listing 11: Output server

```
1 Avvio del Client...
2 Inviato a 127.0.0.1:8080: ping
3 Ricevuto da 127.0.0.1:8080: pong
```

Listing 12: Output client

2 Compilazione

Per compilare entrambi i programmi usiamo `make`.

```
1 make
```

Listing 13: Compilazione

Per rimuovere i file compilati usiamo `make clean`.

```
1 make clean
```

Listing 14: Clean

3 Esercizio 2: File Descriptor

3.1 Obbiettivo

Listare, attraverso l'opportuno tool da riga di comando, il file descriptor relativo al programma precedente ed analizzarne l'output.

3.2 Tool: `lsof`

Il comando `lsof` (List Open Files) permette di visualizzare i file aperti da un processo.

```
1 bash$ lsof -i :8080
```

Listing 15: Comando `lsof`

3.3 Output

1	COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE
	NAME							
2	server_udp	70928	dff	3u	IPv4	334618	0t0	UDP *:
	http-alt							

Listing 16: Output lsof

3.4 Analisi

- **COMMAND:** nome del comando/processo (`server_udp`)
- **PID:** identificativo univoco del processo (70928)
- **USER:** utente che ha lanciato il processo (dff)
- **FD 3u:** file descriptor 3 in modalita' read/write

Contesto sui file descriptor: Ogni processo in Linux ha i suoi file descriptor. I primi 3 sono riservati:

- fd 0: standard input (stdin)
- fd 1: standard output (stdout)
- fd 2: standard error (stderr)

I socket di rete vengono assegnati ai fd successivi. Il nostro server ha il socket su fd 3. La lettera u indica che il socket UDP supporta sia invio che ricezione.

- **TYPE IPv4:** famiglia di indirizzi IPv4
- **DEVICE:** numero del nodo di rete assegnato dal kernel
- **SIZE/OFF:** dimensione del buffer o offset (0t0 = nessun offset)
- **NODE:** numero di porta assegnato dal kernel (334618)
- **NAME:** indirizzo di binding

4 Conclusioni

In questa hands-on ho implementato un sistema client-server UDP funzionante, comprendendo le differenze tra socket UDP e TCP. Ho inoltre appreso come monitorare i file descriptor dei processi di rete utilizzando `lsof`.